TECHNICAL ARCHITECTURE DOCUMENT

Autonomous AI Job Orchestrator

What you have built, where you are, and where you are heading

What this document covers
A plain-language explanation of the system, a full architecture walkthrough, Data Flow Diagrams (Level 0 & 1), an Entity-Relationship diagram, an honest status check against your build plan, and the road ahead.

Project: **autonomous-AI-job-orchestrator**

Branch `main` · 107 tracked files · 25 commits reviewed

Assessed 17 July 2026

Contents

1. The one-paragraph version

You have built a **smart to-do list for computers**.

Programs need to run work: nightly reports, data pipelines, ML training. Somebody has to decide *which piece of work runs next*, make sure it actually runs, retry it if it fails, and keep a record of what happened. That "somebody" is a **job orchestrator**.

Most orchestrators decide the order with a dumb fixed rule — first-come-first-served. Yours uses a **neural network that learned from experience** which job to pick next in order to miss the fewest deadlines. That is the novel bit. Everything else in the project exists to make that idea **safe, durable, and multi-tenant** — i.e. something a real company could actually run.

> **THE HONEST HEADLINE**
>
> The foundation you have built is **genuinely solid**. The hard architectural calls — durable database as the source of truth, at-least-once delivery, AI-as-advisor-with-a-safe-fallback — are all correct and actually implemented, not just written down. You are roughly **40% of the way** to the plan's definition of "real-world ready", and the 40% you did is the *load-bearing* 40%.

2. The restaurant analogy

If you remember one page of this document, make it this one. Every component maps to a role in a restaurant.

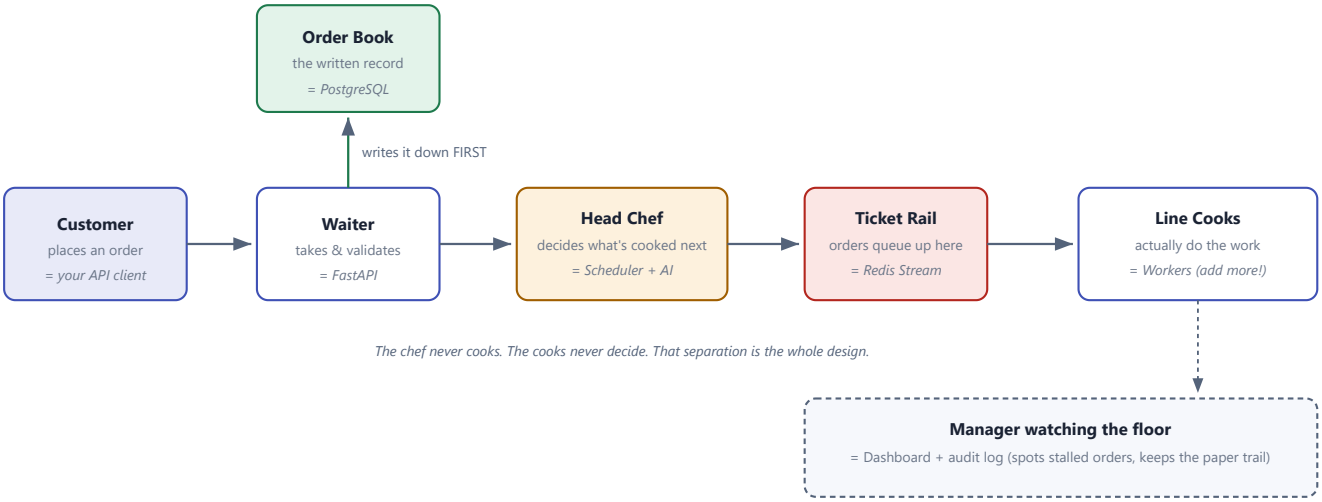


Figure 1 — The system as a restaurant kitchen

Why the separation matters

Two rules drive almost every design decision in your codebase:

- **The waiter writes the order down before telling the kitchen.** If the restaurant burns down and is rebuilt, the orders still exist. This is why Postgres — not Redis — is your source of truth. A job is saved to the database *before* your API returns `201 Created`. Nothing is ever lost.
- **Cooks are interchangeable and disposable.** If a cook collapses mid-dish, another picks up the ticket and redoes it. This is why you have leases, heartbeats, and at-least-once delivery. Losing a worker costs you time, never data.

> **THE KEY INSIGHT YOU SHOULD INTERNALISE**
>
> The AI is the **chef's opinion** — valuable, but the kitchen must work perfectly even if the chef walks out. That is why your scheduler defaults to `SCHEDULER_MODE=heuristic` and only uses the neural network when explicitly switched on. This was the single best decision in the project.

3. Architecture — the component diagram

This is what you have actually built today. Solid boxes exist in code; dashed boxes are planned but not written yet.

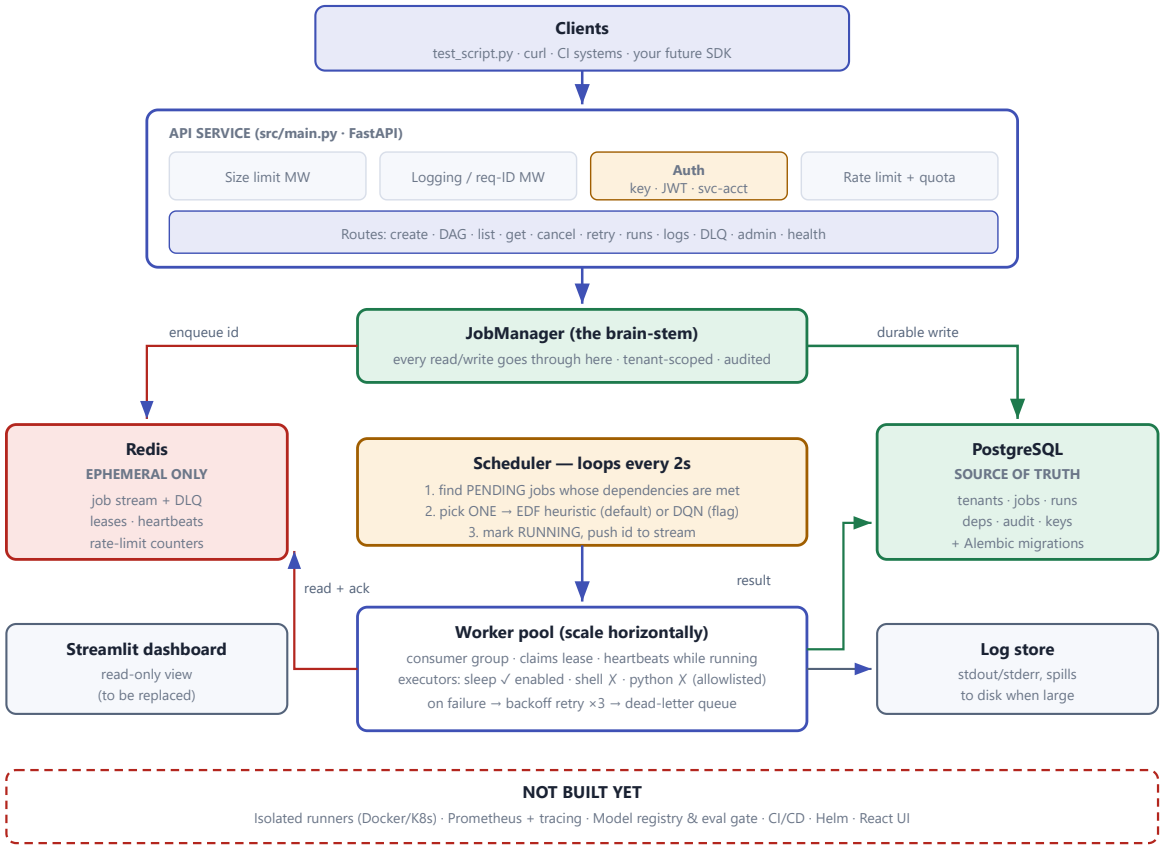


Figure 2 — Current architecture. Green = durable truth, red = ephemeral, amber = decision-making, dashed = not yet built.

> **READ THE ARROWS, NOT THE BOXES**
>
> Notice that **green flows go to Postgres and red flows go to Redis**, and they never swap. Job facts (status, results, history) only ever live in Postgres. Redis only ever holds a *pointer* (the job ID) and short-lived coordination data. If Redis were wiped right now, you would lose queue position — not a single job. That is textbook-correct, and it is the difference between a prototype and a real system.

4. Data Flow Diagram — Level 0 (Context)

A **Level 0 DFD** (also called a context diagram) treats your entire system as a single process and shows only who talks to it and what crosses the boundary. It answers: "*what is inside, what is outside?*"

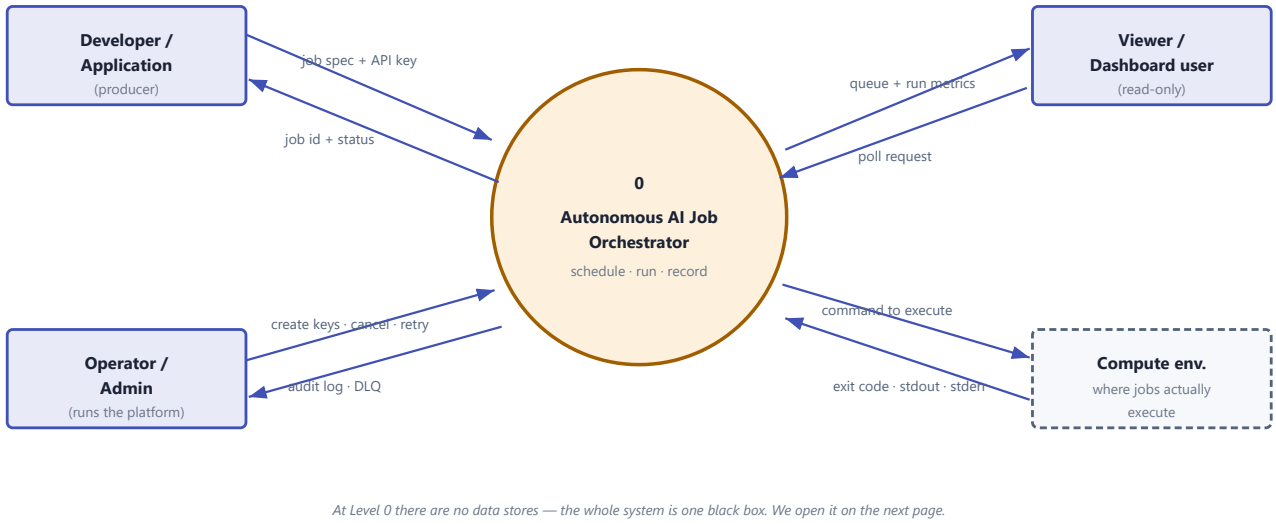


Figure 3 — Level 0 / Context DFD

How to read it

SYMBOL	MEANING
Rectangle	An **external entity** — a person or system outside your boundary. You do not control it.
Circle	A **process** — something your system *does* that transforms data.
Arrow	A **data flow** — labelled with *what* moves, never *how*.
Open rectangle	A **data store** — appears from Level 1 onward (see next page).

Note the four boundary crossings. Three are human/software clients split by **role** — that split is real in your code (`viewer`, `producer`, `operator` in `src/models/auth.py`) and enforced on every route.

5. Data Flow Diagram — Level 1

Now we open the black box. This is the diagram that actually maps to your source files.

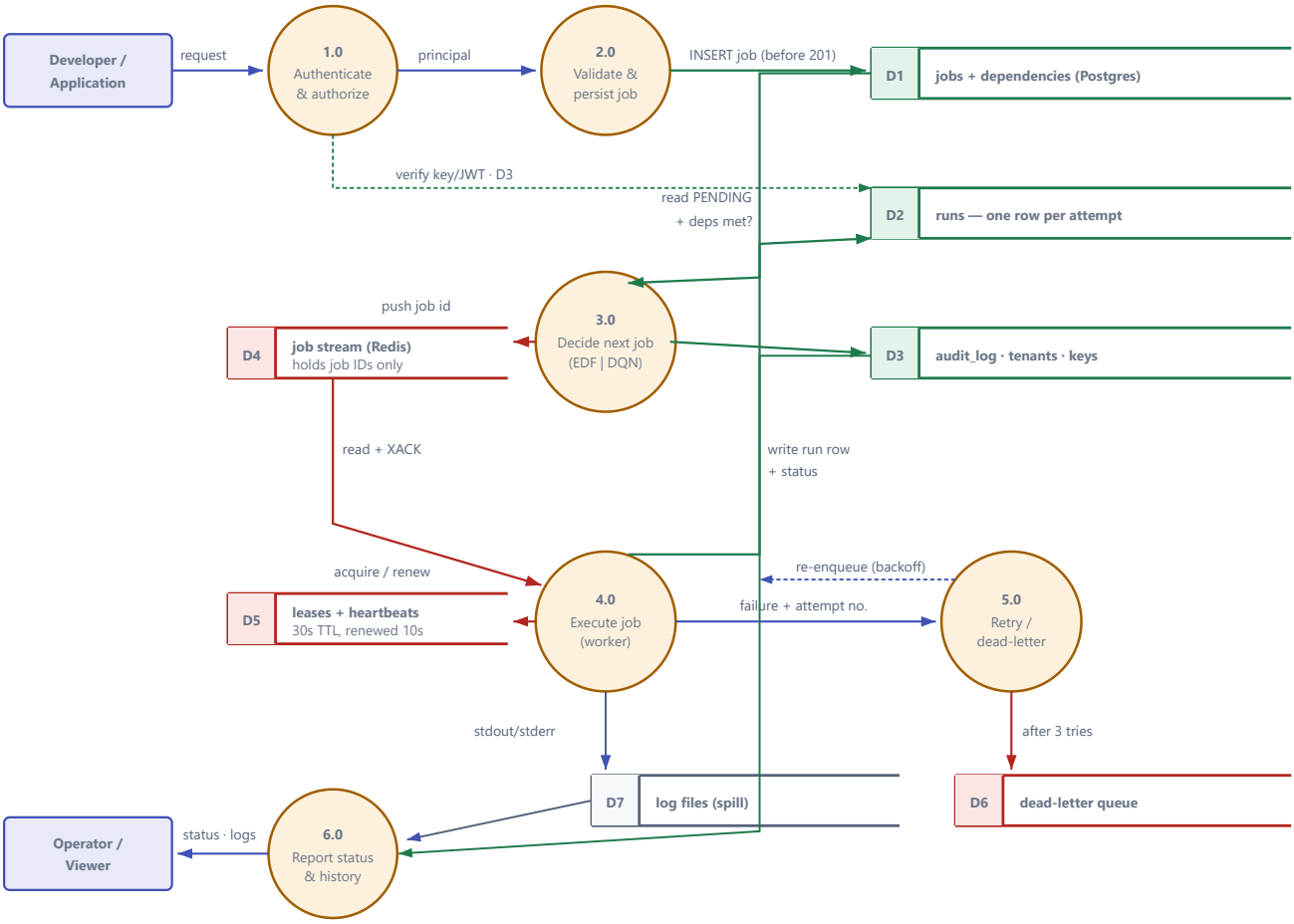


Figure 4 — Level 1 DFD. Green flows touch durable Postgres; red flows touch ephemeral Redis.

PROCESS → SOURCE FILE

#	PROCESS	LIVES IN
1.0	Authenticate & authorize	`src/auth/deps.py`, `service.py`, `jwt_service.py`
2.0	Validate & persist job	`src/api/routes.py` → `job_manager.create_job()`
3.0	Decide next job	`src/orchestrator/scheduler.py`
4.0	Execute job	`src/orchestrator/worker.py` + `executors/`
5.0	Retry / dead-letter	`job_manager.handle_execution_failure()`
6.0	Report status & history	`routes.py` (get/list/runs/logs), `dashboard/app.py`

6. Entity-Relationship Diagram

This is your actual Postgres schema, taken from `src/db/models/`. **PK** = primary key (the unique ID), **FK** = foreign key (a pointer to another table), **UQ** = unique constraint.

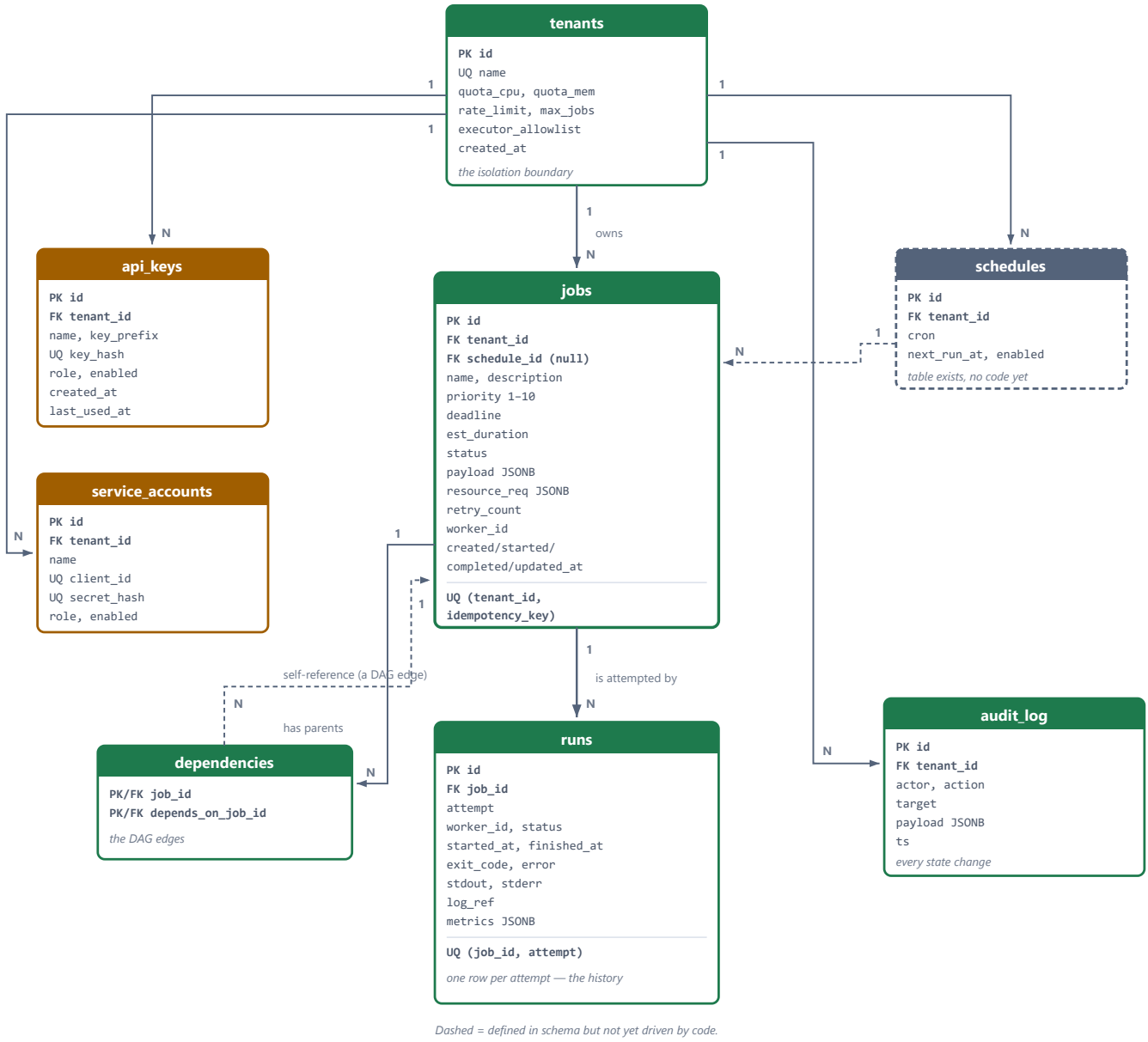


Figure 5 — Entity-Relationship diagram of the live Postgres schema

6b. Your database in plain words

TABLE	WHAT IT IS, AND WHY IT EXISTS
tenants	A customer/organisation. This is your **isolation boundary** — every other table hangs off it, and every query filters by it. It also carries that tenant's limits: how many jobs, how fast they can call you, and which executors they're allowed to use.
jobs	The *intent* — "please run this thing". Note it holds `status`, `priority`, `deadline`, and a JSON `payload` describing the actual work. The unique constraint on `(tenant_id, idempotency_key)` is what makes double-submits safe.
runs	The *attempts*. This is the table most beginners miss. One job can be attempted many times (retries), so each attempt gets its own row with its own exit code, logs, and timing. `UQ(job_id, attempt)` guarantees attempts are numbered cleanly. **Job = what you wanted. Run = what actually happened.**
dependencies	Your DAG. Each row is one arrow: "job_id waits for depends_on_job_id". Both columns point at `jobs`, which makes this a **self-referencing many-to-many** — the standard way to store a graph in a relational database.
api_keys / **service_accounts**	Who is allowed in. Note you store `key_hash`, never the key itself — correct. `key_prefix` exists so a human can recognise a key in a list without it being secret.
audit_log	An append-only record of every state change: who did what, to what, when. Your `JobManager` writes to this on every mutation.
schedules	Cron rules for recurring jobs. table only The table and the FK from `jobs.schedule_id` exist, but nothing writes to it yet. This is honest scaffolding for a feature you planned.

> **WHY THE JOBS/RUNS SPLIT IS THE SMARTEST THING IN YOUR SCHEMA**
>
> If you had stored the exit code and logs on the `jobs` table, a retry would **overwrite** the record of the previous failure — and you'd never be able to answer "why did this fail the first two times?". By splitting them, your failure history is permanent and queryable. This is exactly how Airflow, Temporal, and GitHub Actions model it.

What is deliberately *not* in the database

The queue is not a table. Job IDs waiting to run live in a **Redis Stream**, not Postgres. This is intentional: a database makes a poor queue (polling it hammers the DB), while Redis Streams give you consumer groups, acknowledgements, and redelivery for free. The rule you followed — **Postgres knows the truth, Redis knows the order** — is the right one.

7. The life of one job

Follow a single job from submission to completion. Every step below is real code you have written.

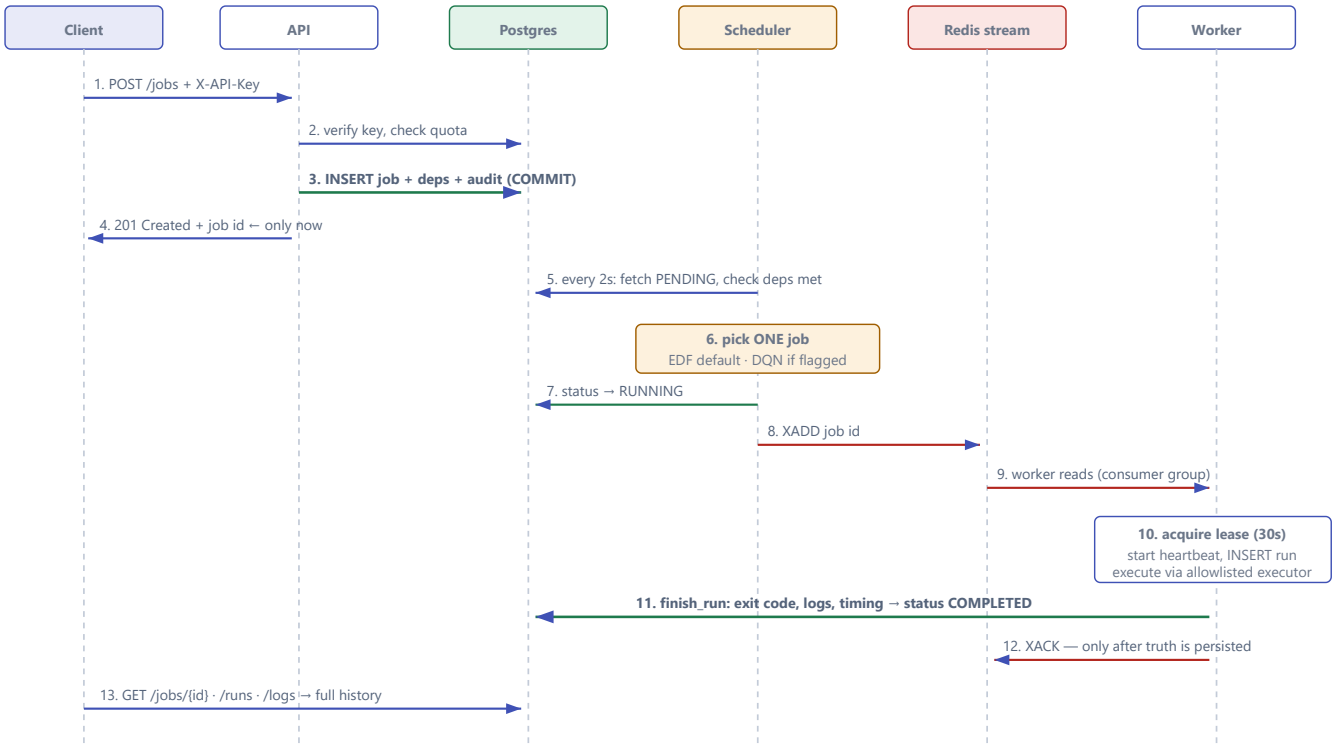


Figure 6 — Sequence: submission to completion

The two steps that make it trustworthy

- **Step 3 before step 4.** The database commit happens *before* the `201` is returned. If your API crashes one millisecond after responding, the job still exists and will still run. This is the "no silent data loss" guarantee from your plan, and you actually honoured it.
- **Step 12 after step 11.** The worker acknowledges the queue message *only after* the result is safely in Postgres. If the worker dies at step 10, nobody acks, the lease expires, and another worker reclaims and reruns it. This is **at-least-once delivery** — and it's why the `UQ(job_id, attempt)` constraint and idempotency keys matter.

8. The AI part, explained simply

Strip away the jargon and the AI does one small thing: **given up to 15 waiting jobs, pick the index of the one to run next.** That's it. It's a chooser.

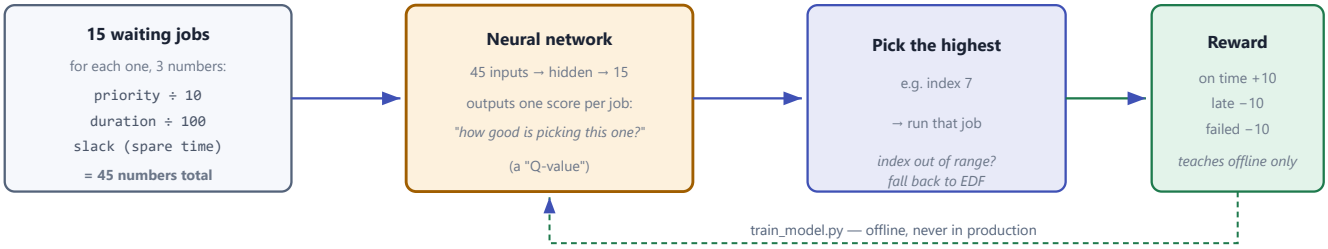


Figure 7 — What the DQN actually computes

Why bother? Because the dumb rules genuinely fail

Your own `benchmark.py` results, over 100 jobs:

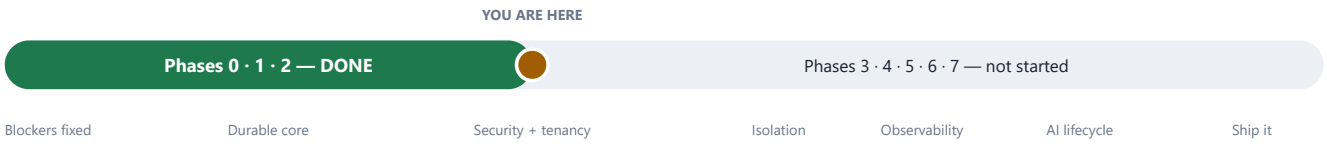
STRATEGY	MISSED DEADLINES	WHAT GOES WRONG
FIFO (first come first served)	34	A long unimportant job blocks an urgent one behind it.
Strict priority	38	**Starvation** — low-priority jobs never run at all, so they all miss.
AI (DQN)	**14**	Learns to balance "important" against "running out of time".

The AI learned something genuinely non-obvious: *sometimes you run the low-priority job first*, because it's about to breach its deadline and the high-priority one still has slack. No simple rule expresses that trade-off well. That's the thesis of your whole project, and the benchmark supports it.

> **BUT IT IS SWITCHED OFF RIGHT NOW — AND THAT IS CORRECT**
>
> Your config ships `SCHEDULER_MODE=heuristic`, so production uses Earliest-Deadline-First, not the AI. Do not "fix" this yet. There is a known **train/serve skew** bug (issue B8 in your plan): `train_model.py` sorts jobs by deadline before feeding the network, but the live scheduler does **not** sort. So the network sees job #3 in training and a totally different job at position #3 in production — the input means something different than what it learned on. Until that is fixed, the 14-vs-34 benchmark number does **not** transfer to production. Fixing this is Phase 5.

9. Where you are right now

I verified this against your code, not your commit messages. Here is the honest map.



PHASE	STATUS	EVIDENCE IN YOUR CODE
0 Stabilize	DONE	All 10 blockers fixed. `shell.py` duplicate-timeout gone + output truncated; worker no longer double-completes; dangerous executors allowlisted off; scheduler runs `epsilon=0`, eval-mode, no live training; rewards unified in one file; dead files deleted; structured logging in.
1 Durable core	DONE	Postgres + SQLAlchemy + 4 Alembic migrations. Redis Streams with consumer groups and explicit `XACK`. Leases + heartbeats + stale-message reclaim. Retries with exponential backoff → DLQ. Idempotency keys. Run logs that spill to disk. Cancel/retry/paginate/runs/logs/DAG endpoints.
2 Security	DONE	API keys + JWT + service accounts. Three roles enforced per-route. Tenant scoping on every query. Quotas, rate limits, body-size limits. Audit log on every mutation. Config refuses to boot in production with default secrets.
3 Isolation	Not started	Jobs still run inside the worker's own process. A heavy job can still hurt its worker.
4 Observability	Not started	Zero Prometheus/OpenTelemetry code exists. You have logs and health endpoints, no metrics or traces.
5 AI lifecycle	Not started	No replay buffer, no target network, no model registry, no eval gate. B8 skew still open.
6 Delivery	Not started	No CI, no Helm/K8s/Terraform, no ruff/mypy config, UI still Streamlit.
7 Go-live	Not started	Depends on everything above.

WHAT THIS MEANS

You did the phases **in the right order** and you did not skip ahead to the fun parts. Most people build the React dashboard before they've made data durable. You made data durable first. The remaining phases are all *additive* — they bolt onto what you have rather than forcing a rewrite. That is the clearest possible sign the architecture is sound.

10. What is broken today

Three real problems. The first one needs fixing before anything else.

> **BLOCKER — AUTHENTICATION IS BROKEN IN MAIN RIGHT NOW**
>
> `src/models/auth.py` defines `class Principal` **twice** — once at line 22 (with field `api_key_id`) and again at line 59 (with field `subject_id`). Python silently keeps the second one. But two places still build it the old way: `src/auth/service.py:103` and `src/auth/deps.py:27`.
>
> Result: every authenticated request raises `ValidationError: Field required [subject_id]`. This is not a test-only problem — **API-key login does not work at all**. It arrived with your most recent commit (`b868f24 adding oauth and JWT`), which renamed the field but only updated some of the callers.
>
> **Fix:** delete the dead first definition, change the two call sites to `subject_id=`, keep the `api_key_id` property as a read alias.

Test suite: 19 failed, 31 passed

FAILURE	COUNT	CAUSE
`test_api`, `test_tenant_isolation`	16	The `Principal` blocker above. All one root cause.
`test_db_schema`	2	Environmental — needs a live Postgres. Should be marked as an integration test and skipped by default.
`test_executor_policy`	1	Cosmetic — the test expects the string "disabled by policy", the code now says "disabled by tenant policy". The security behaviour is correct; the assertion is stale.

The scheduler will not scale (worth knowing before Phase 3)

Three issues, all inside the 2-second loop in `src/orchestrator/scheduler.py`:

1. **You are capped at 0.5 jobs per second.** The loop picks exactly *one* job, enqueues it, then sleeps 2 seconds. Adding workers cannot raise throughput — the bottleneck is upstream of them. This quietly contradicts your README's "just add more workers" claim. Fix: enqueue a batch per tick.
2. **It scans every job, every 2 seconds.** `list_jobs(limit=10_000)` pulls all jobs into Python and filters for PENDING in memory. Your `list_jobs` already accepts a `status` filter — just use it and let Postgres do the work.
3. **N+1 queries on dependency checks.** `are_dependencies_met` opens a fresh database session for *every parent of every job*, every tick. Fix: one SQL query with a `NOT EXISTS`.

> **ALSO WORTH FLAGGING**
>
> `scheduler = Scheduler()` runs at *import time* (line 83). If you ever scale the API to more than one replica, you get multiple schedulers all enqueueing the same jobs. You'll need leader election before you scale out. Not urgent today; important at Phase 3.

11. Where you are heading

Right now — this week

1. **Fix the `Principal` duplicate.** Auth is broken in `main`. Nothing else matters until this is green.
2. **Get `pytest` green.** Fix the stale assertion; mark the Postgres tests as integration and skip them without a database.
3. **Add CI immediately.** This is listed as Phase 6 in your plan — **pull it forward to now**. A GitHub Action running `pytest` on every push would have caught the auth blocker before it reached `main`. It is roughly 20 lines of YAML and it is the highest-leverage thing you can do today.

Next — Phase 3, isolation

Fix the three scheduler scaling issues above *first* (they're cheap and they're in the hot path), then move job execution out of the worker process into subprocesses with real CPU/memory limits. This is what lets you safely re-enable the `shell` and `python` executors — which is what makes the platform actually useful for real workloads rather than `sleep` demos.

Then — Phase 4, observability

Prometheus metrics on queue depth, scheduling latency, and missed deadlines. This is worth doing *before* the AI work, for a specific reason: **you cannot safely roll out a model you cannot measure.** Shadow-mode comparison in Phase 5 needs the metrics from Phase 4 to be meaningful.

Then — Phase 5, the AI lifecycle

This is where your project's actual thesis gets proven. In order:

- Fix **B8** — make one shared encoding function that both the trainer and the scheduler call, so they cannot drift apart. Sort consistently in both.
- Add a **replay buffer** and a **target network** — standard DQN machinery that makes training stable and reproducible.
- Build the **eval gate**: a new model must beat FIFO and EDF on held-out scenarios before it can be promoted. Automate it.
- Roll out in **shadow mode** first — AI suggests, heuristic decides, log where they disagree. Only then canary, then full, always with the kill switch.

Finally — Phases 6 & 7

Helm chart, React UI, CLI/SDK, then the dogfood → pilot → beta → GA ladder in your plan.

> **THE ONE STRATEGIC NOTE**
>
> Your plan schedules CI at Phase 6, and that is the single ordering mistake in an otherwise well-sequenced document. Every phase from here adds more code to a system whose correctness you currently verify by hand. The auth blocker sitting in `main` right now *is* the cost of that gap, and it will not be the last one. Move CI to now.

Answering your actual question

Are you heading the right way? Yes. Unambiguously. The architecture is sound, the phase ordering is disciplined, and the parts you have finished are the parts that are expensive to retrofit later. Durable-truth-before-acknowledgement, at-least-once with leases, tenant isolation on every query, and AI-as-advisor-with-a-fallback are decisions that experienced platform engineers get wrong regularly. You got them right. Fix the auth blocker, add CI, and keep going in the order you planned.

12. Glossary

TERM	PLAIN MEANING
Orchestrator	The thing that decides what work runs, when, and makes sure it finished.
Job	The *request* — "please run this". Stored once.
Run	One *attempt* at a job. A job that retried 3 times has 3 runs.
DAG	Directed Acyclic Graph — jobs with arrows: "C waits for B waits for A". Acyclic = no loops (or nothing could ever start).
Source of truth	The one place where a fact is authoritative. Yours is Postgres. If two systems disagree, the source of truth wins.
Redis Stream	A queue with memory. Unlike a simple list, readers form a *consumer group*, must *acknowledge* messages, and unacked messages get redelivered.
XACK	"I'm done with this message, stop tracking it." Sent only after your work is durably saved.
Lease	A time-limited claim: "I own job X for the next 30 seconds." Stops two workers running the same job.
Heartbeat	"I'm still alive, extend my lease." Stops when the worker dies, so the lease expires and someone else takes over.
At-least-once	A guarantee that work runs, possibly more than once (if a worker dies mid-way). The alternative, at-most-once, can silently lose work. At-least-once + idempotency is the standard trade.
Idempotency key	A client-supplied ID meaning "if you've already seen this, don't create a duplicate." Makes retrying a submission safe.
DLQ	Dead-Letter Queue — where jobs go after exhausting retries, so a poison job can't loop forever but also isn't silently dropped.
Multi-tenancy	One deployment serving multiple isolated customers. Tenant A must never see tenant B's jobs.
Alembic migration	A versioned, replayable script that changes the database schema — so you can upgrade production without losing data.
EDF	Earliest-Deadline-First. The simple, predictable rule your scheduler uses by default: run whatever is due soonest.
DQN	Deep Q-Network. A neural net that scores each possible choice by expected future reward, and you take the highest.
Q-value	"If I pick this option now, how much total reward do I expect from here on?"
Epsilon	How often the agent picks randomly to explore. High while learning, **zero** in production. Yours is 0. Correct.
Replay buffer	A memory of past experiences that training samples randomly, so the net doesn't overfit to whatever just happened. You don't have one yet.
Target network	A frozen copy of the net used as a stable reference during training. Without it, training chases its own tail. You don't have one yet.

Train/serve skew	When the data a model sees in production is shaped differently from its training data. You have this (issue B8) — it's why the AI stays off.
Shadow mode	The new model runs and logs its choices, but the old system still decides. Free real-world evidence with zero risk.
Starvation	When low-priority work never runs because something more important always jumps ahead. Your benchmark shows strict-priority scheduling causing exactly this.
Slack	Spare time: how long until the deadline, minus how long the job takes. Negative slack = already too late.